Eficiencia de algoritmos: O(n) frente a O(n²)

Actividad de análisis de complejidad algorítmica

1. Cinco fragmentos, cinco complejidades

Fragmento de código	Pasos aproximados	Notación	¿Por qué?
Leer vector[5]	1	O(1)	Solo se toca una posición, sin importar qué tan grande sea el arreglo.
Pasar por todo el arreglo	n	O(n)	Un ciclo visita cada elemento una única vez, así que el trabajo aumenta a la par de n.
Dos ciclos uno dentro de otro	$n \times n = n^2$	$O(n^2)$	Por cada vuelta del ciclo de afuera, el de adentro repite sus n vueltas completas.
Dividir n entre 2 una y otra vez	$\log_2 n$	O(log n)	Cada paso deja el problema en la mitad, por eso la cantidad de pasos sube muy despacio.
Ciclo con una reducción a la mitad adentro	$n \times \log_2 n$	O(n log n)	El ciclo exterior corre n veces y en cada vuelta se hacen cerca de log n divisiones (igual que en el Ejercicio 5).

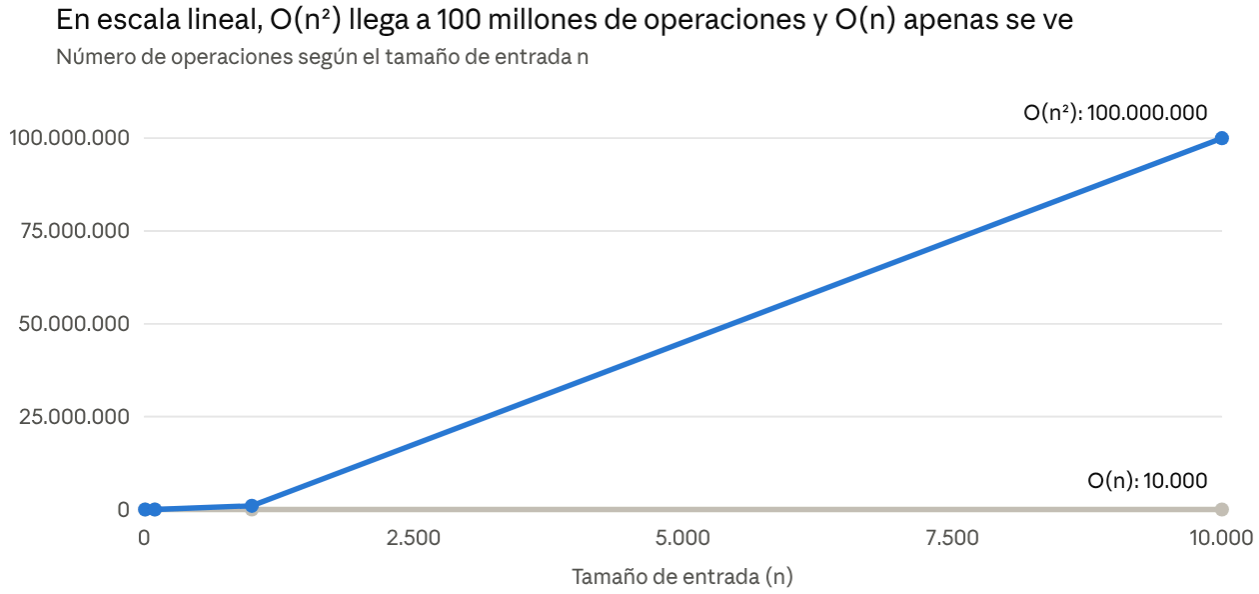
2. Tiempos que medí en el computador

Todos los valores están en nanosegundos (ns).

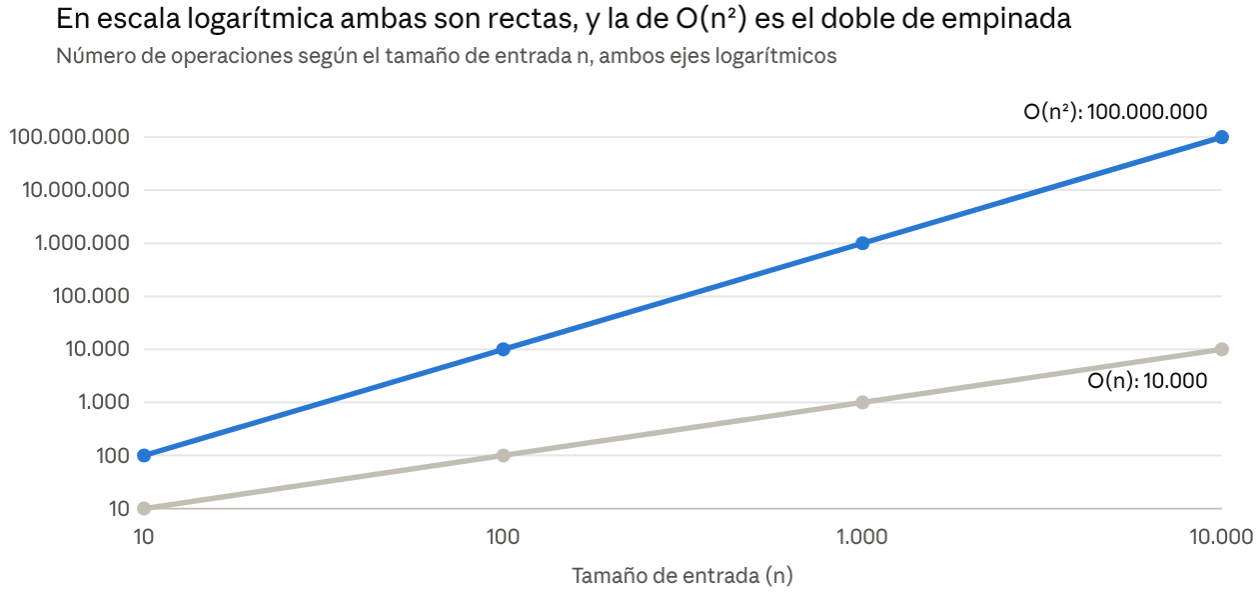
Tamaño n	Tiempo de O(n)	Tiempo de O(n²)	Algoritmo más veloz
100	6.700	362.700	O(n)
1.000	42.800	8.953.900	O(n)
5.000	162.000	6.618.100	O(n)
10.000	316.900	14.656.200	O(n)

3. Cómo crece el número de operaciones

n	O(n)	O(n²)	Cuántas veces mayor es O(n²)
10	10	100	10
100	100	10.000	100
1.000	1.000	1.000.000	1.000
10.000	10.000	100.000.000	10.000



Calculado a partir de la tabla de la sección 3 (n = 10, 100, 1.000 y 10.000)



Calculado a partir de la tabla de la sección 3 (n = 10, 100, 1.000 y 10.000)

Lo que se puede observar

1. Si n pasa de 10 a 100, O(n) se hace 10 veces más grande, mientras que O(n²) se hace 100 veces más grande.
2. Para n = 10.000, el caso lineal requiere apenas 10.000 operaciones, pero el cuadrático necesita 100.000.000.
3. Al dividir O(n²) entre O(n) siempre se obtiene n, o sea que la distancia entre ambos se abre más a medida que n aumenta.
4. En la gráfica de escala lineal, O(n) casi no se despega del eje horizontal y O(n²) sale disparada hacia arriba; ahí se ve el crecimiento explosivo.
5. En la gráfica logarítmica las dos curvas se convierten en rectas, y la de O(n²) es el doble de empinada que la de O(n).
6. Un algoritmo O(n²) no escala bien: cuando hay muchos datos es mejor buscar una alternativa O(n) o O(n log n).

4. Preguntas de análisis

1. ¿En qué se diferencian el tiempo de ejecución y la complejidad Big O?

El tiempo de ejecución es lo que se mide en el computador (en mi caso, en nanosegundos) y cambia según el procesador, la memoria y los demás procesos que estén corriendo. Big O no mide segundos: explica cómo crece la cantidad de operaciones cuando n aumenta, y por eso no depende de la máquina.

2. ¿Por qué O(1) se considera eficiente?

Porque el trabajo es el mismo sin importar cuántos datos haya. Leer datos[0] cuesta igual en un arreglo de 10 elementos que en uno de un millón.

3. ¿Qué le pasa a un algoritmo O(n) cuando se duplica el tamaño?

Su tiempo también se duplica, más o menos. En mi prueba, al pasar de n = 5.000 a n = 10.000 el tiempo subió de 162.000 a 316.900 ns, alrededor de 1,96 veces: casi el doble.

4. ¿Y a uno O(n²)?

En teoría el trabajo se multiplica por cuatro, porque $(2n)^2 = 4n^2$. En lo que medí, de 5.000 a 10.000 el tiempo pasó de 6.618.100 a 14.656.200 ns, cerca de 2,2 veces. No llegó a 4 porque las mediciones traen ruido (la JVM todavía se está calentando, hay optimizaciones y otros procesos de por medio), pero igual se nota que crece mucho más rápido que el caso lineal.

5. ¿Qué relación hay entre los ciclos anidados y O(n²)?

Por cada una de las n vueltas del ciclo externo, el interno hace otras n. Eso da n × n = n² operaciones. En el Algoritmo B, con n = 10.000 se realizan 100.000.000 de incrementos.

6. ¿Cómo se reconoce un algoritmo O(log n)?

Porque en cada paso el problema se recorta a la mitad (o a una fracción), como en n = n / 2. Por eso el número de pasos sube con mucha lentitud.

7. ¿Cuál de los dos algoritmos creció más?

El Algoritmo B, que es O(n²). Entre n = 100 y n = 10.000, el tiempo de O(n) pasó de 6.700 a 316.900 ns (unas 47 veces), y el de O(n²) pasó de 362.700 a 14.656.200 ns (unas 40 veces en lo medido, aunque en operaciones teóricas son 10.000 veces más). En todos los tamaños el cuadrático fue el más lento: con n = 10.000 se demoró unas 46 veces más que el lineal.

8. ¿Los resultados fueron idénticos en cada ejecución?

No. Los tiempos cambian por el procesador, la memoria disponible, el sistema operativo, los procesos abiertos y las optimizaciones de Java (JIT). En mis datos se nota: O(n²) con n = 1.000 tardó 8.953.900 ns y con n = 5.000 tardó 6.618.100 ns, cuando en teoría 5.000 debería tomar bastante más. Seguramente fue por el calentamiento de la JVM en las primeras corridas. Por eso no hay que confundir el tiempo medido con la complejidad Big O.

9. Si tuviera que procesar diez millones de registros, ¿qué complejidad elegiría?

O(n) u O(n log n). Con n = 10.000.000, un algoritmo O(n²) pediría cerca de 10^{14} operaciones. Con la velocidad que medí en mi computador (unos 14,6 ms por cada 10^8 operaciones), eso serían varias horas, mientras que uno O(n) quedaría listo en una fracción de segundo.

10. ¿Por qué un ingeniero debe analizar la eficiencia antes de escoger un algoritmo?

Porque un algoritmo puede andar bien con pocos datos y volverse inservible con muchos, como pasó con O(n²). Una mala elección afecta el tiempo de respuesta, el uso de recursos y los costos. Que funcione no es suficiente: hay que saber cómo se va a comportar cuando la información crezca.

5. Conclusiones

Antes de esta actividad yo pensaba que si un programa entregaba el resultado correcto ya estaba bien. Ahora entiendo que eso es solo una parte del trabajo. Con n = 100 ambos algoritmos terminaron en menos de un milisegundo, pero con n = 10.000 el cuadrático tomó 14.656.200 ns y el lineal apenas 316.900 ns, unas 46 veces menos. Si con diez mil datos ya se siente esa diferencia, con millones sería un problema serio.

Una cosa que me sorprendió fue que los tiempos no siguieron al pie de la letra lo que dice la teoría: el cuadrático con n = 1.000 tardó más que con n = 5.000. Comprendí que lo medido depende del computador, de la memoria e incluso de cómo la JVM optimiza el código, y que por eso Big O no habla de segundos sino de cómo aumenta el trabajo cuando n crece.

Me quedo con la idea de que antes de elegir un algoritmo hay que pensar en cuántos datos va a manejar. Un código que hoy funciona puede quedar inútil cuando la aplicación crezca. Prefiero analizar primero que descubrir el problema cuando ya es tarde.